AL-2002 - Artificial Intelligence Lab
Lab # 4

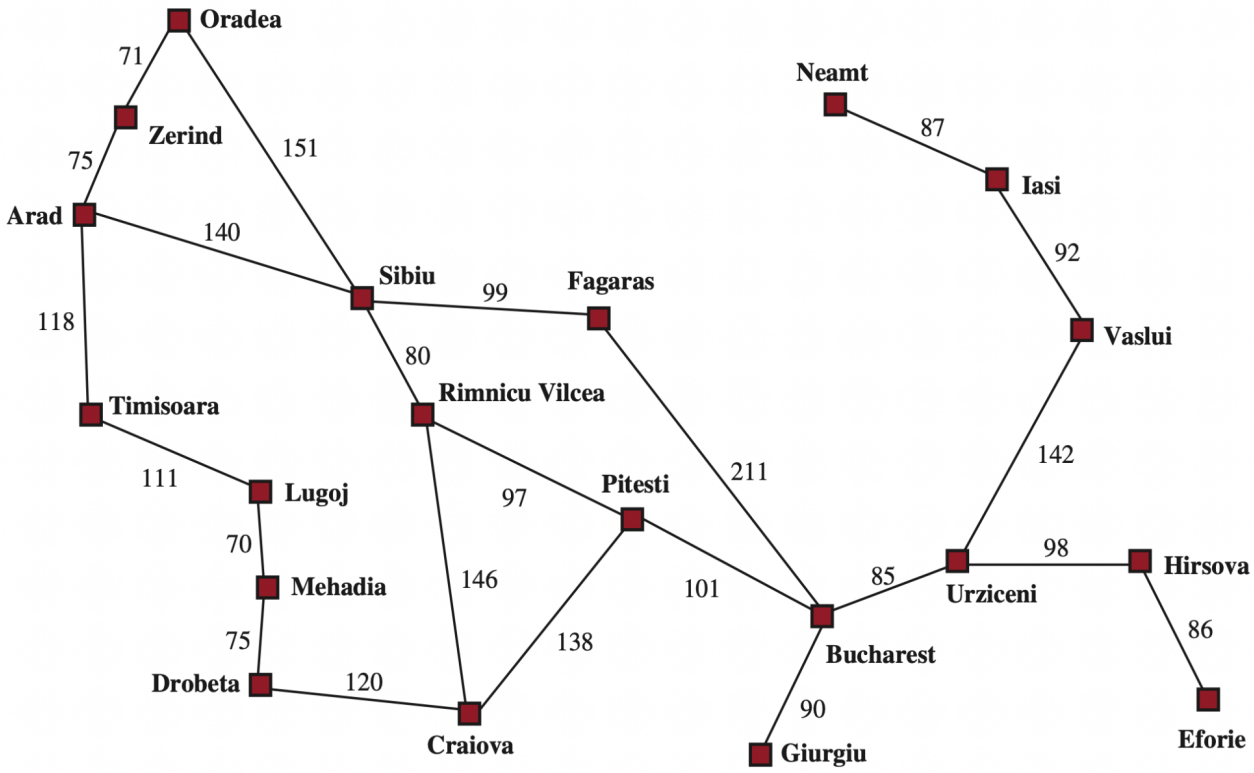
Objectives:

- Informed Searches (A*) + Adversarial Search (MinMax + AlphaBeta Pruning)

Note: Carefully read the following instructions (*Each instruction contains a weightage*)

1. First think about statement problems and then write your program.
2. Write Program in multiple cells of the same notebook**.**
3. **Must Use Functions and Exception handling for each question, where they can be applied.**
4. Do not copy from any source otherwise you will be penalized with negative marks.
5. Add your source code in word document and attach output screenshots.
6. Upload word file and ipynb file.
7. Please submit your **Both files** with this naming convention ROLLNO_SECTION_LABNO. **(Do not upload zip file).**
8. Submit your lab on Google Classroom.

Question 1:



City	h(n) to Bucharest
Arad	366
Zerind	374
Oradea	380
Sibiu	253
Timisoara	329
Lugoj	244
Mehadia	241
Drobeta	242
Craiova	160
Rimnicu Vilcea	193
Fagaras	176
Pitesti	100
Bucharest	0
Giurgiu	77
Urziceni	80
Hirsova	151
Eforie	161
Vaslui	199
Iasi	226
Neamt	234

A smart navigation system needs to find the most efficient route between two Romanian cities not just the route with fewest stops, and not just the cheapest road distance, but the route that balances **actual travel cost** with an **estimated remaining distance to the destination**.

Using the Romania road map, implement **A* Search** to find the optimal path from a given source city to a destination city. A* uses the evaluation function:

f(n) = g(n) + h(n)

Where:

- **g(n)** = actual cumulative road distance from source to current city
- **h(n)** = estimated straight-line distance from current city to destination (provided in the heuristic table below)
- **f(n)** = total estimated cost of the path through current city

The algorithm must always expand the city with the lowest **f(n)** value next. Your program should display the city being explored at each step along with its g(n), h(n) and f(n) values, the final path discovered, and the total actual road distance of that path.

Question 2:

```
function MINIMAX-SEARCH(game, state) returns an action
  player ← game.TO-MOVE(state)
  value, move ← MAX-VALUE(game, state)
  return move

function MAX-VALUE(game, state) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v ← −∞
  for each a in game.ACTIONS(state) do
    v2, a2 ← MIN-VALUE(game, game.RESULT(state, a))
    if v2 > v then
      v, move ← v2, a
  return v, move

function MIN-VALUE(game, state) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v ← +∞
  for each a in game.ACTIONS(state) do
    v2, a2 ← MAX-VALUE(game, game.RESULT(state, a))
    if v2 < v then
      v, move ← v2, a
  return v, move
```

Figure 5.3 An algorithm for calculating the optimal move using minimax—the move that leads to a terminal state with maximum utility, under the assumption that the opponent plays to minimize utility. The functions MAX-VALUE and MIN-VALUE go through the whole game tree, all the way to the leaves, to determine the backed-up value of a state and the move to get there.

In two-player adversarial games, the Minimax algorithm is used to determine the optimal move for a player assuming that the opponent also plays optimally. In this problem, consider a two-player turn-based game tree where the MAX player moves first and the opponent is the MIN player. The game tree has a depth of 3, and the terminal nodes contain the following utility values for the MAX player: **[3, 5, 6, 9, 1, 2, 0, -1].** Write a Python program to implement the Minimax search algorithm that recursively explores the game tree and determines the optimal move for the MAX player. Your program should include functions corresponding to MAX-VALUE and MIN-VALUE, identify terminal nodes, and propagate utility values from the leaf nodes up to the root according to the Minimax decision rule. The program should display the utility value at each decision level, determine the best move from the root node, and print the optimal utility value obtained by the MAX player.

Question 3:

```
function ALPHA-BETA-SEARCH(game, state) returns an action
  player ← game.TO-MOVE(state)
  value, move ← MAX-VALUE(game, state, −∞, +∞)
  return move

function MAX-VALUE(game, state, α, β) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v ← −∞
  for each a in game.ACTIONS(state) do
    v2, a2 ← MIN-VALUE(game, game.RESULT(state, a), α, β)
    if v2 > v then
      v, move ← v2, a
      α ← MAX(α, v)
    if v ≥ β then return v, move
  return v, move

function MIN-VALUE(game, state, α, β) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v ← +∞
  for each a in game.ACTIONS(state) do
    v2, a2 ← MAX-VALUE(game, game.RESULT(state, a), α, β)
    if v2 < v then
      v, move ← v2, a
      β ← MIN(β, v)
    if v ≤ α then return v, move
  return v, move
```

Figure 5.7 The alpha–beta search algorithm. Notice that these functions are the same as the MINIMAX-SEARCH functions in Figure ??, except that we maintain bounds in the variables α and β, and use them to cut off search when a value is outside the bounds.

Consider a two-player game tree of depth 3 **where the** MAX player starts the game **and players alternate turns between** MAX and MIN**. The** terminal nodes contain the following utility values for the MAX player**, evaluated from** left to right**: [5, 6, 7, 4, 5, 6, 7, 8]. Write a** Python program **to implement the** Minimax algorithm with Alpha–Beta pruning **to determine the** optimal move for the MAX player**. Your program should recursively evaluate the game tree, maintain and update the** α (alpha) and β (beta) values**, and** prune branches when possible **during the search. The program should display the** optimal utility value at the root node**, indicate** which branches are pruned**, and print the** best move selected by the MAX player**. Use** functions, recursion, and appropriate exception handling **where applicable.**